ADR 0002 — Tool-call schema and contract

Status: Proposed (Marv to confirm) **Date**: 2026-06-20 **Author**: Marv **Related**: ADR 0001, ADR 0007, ADR 0009, src/baymax/core/schemas/tool_call.schema.json, docs/DEFERRED.md

---

Context

BAYMAX's agent core emits tool calls. Ronin's eval harness consumes them and scores them. The contract between agent core and eval harness — and the internal shape of a tool-call within the agent — defines what both modules must produce/consume to integrate.

Ronin's `src/baymax/eval/scorer.py` already locks the boundary shape:

```
class AgentResponse:
    tool_calls: list[AgentToolCall]
    message: str | None

class AgentToolCall:
    tool: str  # namespaced: "^[a-z][a-z0-9_]*\.[a-z][a-z0-9_]*$"
    arguments: dict[str, Any]
```

The agent core's contract must serialise to this shape at the boundary. Internally, the agent works with a richer `task_file` structure containing context, validation results, action log, cost, etc.

Decision

The tool-call contract defines two distinct shapes:

1. External (boundary) shape — `AgentResponse`

This is what the agent core emits to Ronin's eval runner. **It matches** `scorer.AgentResponse` **exactly** to avoid serialisation drift.

```
{
  "tool_calls": [
    {
      "tool": "calendar.create_event",
      "arguments": {
        "title": "Meeting with Sarah",
        "start": "2026-06-21T09:00:00Z",
        "duration_minutes": 30
      }
    }
```

```
  ],
  "message": "Booked 30-min meeting with Sarah tomorrow at 9am UTC."
}
```

Rules:

- `tool` must match `^[a-z][a-z0-9_]*\.[a-z][a-z0-9_]*$` (namespaced lowercase)
- For clarification or refusal scenarios: `tool_calls` is empty `[]`, `message` contains the clarification question or refusal reason
- For multi-step scenarios: `tool_calls` has multiple entries in execution order

2. Internal shape — `task_file`

This is the agent's internal representation of a request. It is **never serialised to the eval boundary** — it stays inside the agent.

Input fields (what the agent receives or constructs from the user request):

```
task_id           hierarchical, format <run_id>.<task_int>.<phase_int> (Phase 1: all <run_id>.N.0)
request_text      original user input
input_type        enum: text, voice
request_type      enum: validation, creation, edit, delete
request_intent    nullable string (model-generated; null → triggers clarification)
request_complexity enum: single_tool, multi_tool, multiphase (multiphase deferred to v2)
context           opaque dict (datetime, location, file refs, prior task context for sub-tasks)
timeout_cap       expected max number of tool calls (budget; soft limit)
available_tools   list of tool names (matches Ronin's available_tools)
```

Output fields (what the agent populates as it runs):

```
response_text          natural-language response to user
request_completion_status  enum: complete, clarification_pending, aborted, failed
time_elapsed_ms        wall-clock duration
cost                   RunCost reference (see classes refactor below)
action_log             list of (tool_call, result) tuples in execution order
response_type          enum: task_done, clarification, task_refusal, task_ongoing
                       (task_success is NOT in this enum — see "task_done vs task_success" below)
tools_called           list of tool names in execution order
tool_call_success      list of booleans, parallel to tools_called
task_id                same as input
```

3. Error types

```
auth_error          adapter could not authenticate (token expired, missing env var)
type_error          argument failed schema validation (wrong type, missing required field)
ambiguous_request   model could not determine intent → triggers clarification
intent_error        model produced internally-inconsistent plan (hallucinated tool, contradictory
```

```
steps)
timeout_error       tool call exceeded timeout_cap or wall-clock budget
```

4. task_done vs task_success

- **task_done**: agent finished without throwing — produced an `AgentResponse` and terminated cleanly. Set by the agent core.
- **task_success**: the EXPECTED OUTCOME actually happened in the world — verified by Ronin's eval scorer. **Always null inside the agent**; populated post-hoc by the eval harness.

This distinction lets the system distinguish "agent ran cleanly but the user's real intent wasn't satisfied" (hallucinated success) from "agent crashed."

5. Verification gate

Each tool adapter declares `reversible: bool` (and implements `undo_action()` if claiming reversibility, per ADR 0009). For tools where `reversible == False`, a verification gate is automatically applied: the agent must obtain user confirmation before the tool fires. v1 fake adapters are all reversible; verification gates therefore don't trigger in v1 eval. Real-adapter integration (v2) will surface this.

6. Conventions

- **Object nesting**: limited to 3 levels deep in tool `arguments` (prevents pathological JSON dependencies)
- **Idempotency window**: 1 day default; configurable per tool
- **Duplicate requests**: for duplicate tool API calls, return the previous report. For duplicate user request, change `request_completion_status` to `clarification` and `request_type` to `ongoing` (asks the user "did you mean to repeat?")
- **Timeouts**: error message includes Action log + pinpoint where the error occurred. For multi-tool tasks, the `last_successful_tool_called` variable is added to context on retry so the model can resume from the failure point with full awareness of what already worked
- **Auth**: tool adapters read credentials from environment variables. Adapters NEVER expose credentials in returned data or trace logs. Telemetry redacts any field matching `auth*`, `token*`, `key*`, `password*` (see ADR 0010 telemetry)

7. Argument matching convention

For Phase 1, the `_contains` suffix convention used in scenario expected arguments (e.g., `body_contains: "thanks"` matches `body: "many thanks"`) is **eval-side only**. The agent emits plain values. Ronin's scorer handles substring matching when scenarios declare `_contains` expectations. v2 may revisit whether the agent itself should be able to emit `_contains` outputs.

Reasoning

The user-facing and model-facing sides of the agent each have different reasoning and different utility for the evaluation harness, so splitting between them is my way of organising to reduce coupling in the eval — each side should be evaluable independently. Outward fields need to match ahmed's scorer expectations so the boundary contract stays stable across runs and we don't get serialisation drift between the agent and the scorer. Inward fields exist to extract the most information from the user request and standardise it for cross-

model review, so GPT-4, Claude, and the fine-tuned Qwen all consume the same shape and the comparison stays apples-to-apples. Internal fields collect runtime state — cost, action history, error triggers, and a systemic trace of how the model worked — which gives ahmed material to work with during the Qwen fine-tuning stage and also lets us isolate the "agent claims success but the world didn't change" failure mode by separating `task_done` (agent's claim) from `task_success` (predicate-verified outcome).

Alternatives considered

Alternative A — Single flat schema (no internal/external split)

Use one schema for both the boundary contract AND the agent's internal state. why rejected - the internal agent needs richer state (action log, cost, intermediate context) that doesn't belong at the boundaryyet for v1 but will be of use in v2+ expansion, may as well implement them now and change the response shape later; forcing both to share a shape would either bloat the boundary with useless stuff or limit the internal state.

Alternative B — Match OpenAI / Anthropic native tool-use shape exactly

Adopt the vendor SDK's tool-call response shape (e.g., OpenAI's `ChatCompletionMessageToolCall`) as the boundary contract.

why rejected - it would be provider locked-in; Qwen via MLX doesn't emit the same shape natively for example; ahmeds's scorer already locked a different (cleaner) shape; standardising on ahmed's shape gives us one transformation point rather than per-provider adapters everywhere. easier to run unbaised evaluations for baseline.

Alternative C — Include expected-state predicates in the contract

Have the agent emit "expected world state after this call" alongside each tool call, for the eval to verify.

why rejected - ahmed's v1 scorer is tool-call-comparison-based, not state-predicate-based; adding predicates would require the agent to predict effects of every tool call which is more work for no v1 benefit; if ahmed's scorer ever switches to state-predicate evaluation, which affect phase 1 meeting is confirmed to not happen, (but his SCORING.md flags this as future work tho, talk to him about this), this ADR can be revisited.

Consequences

Positive

- Clean separation: external boundary stays minimal (matches Ronin's scorer), internal state can evolve without breaking eval
- The `task_done` / `task_success` distinction architecturally supports the "hallucinated success" failure mode that Ronin's scorer expansion plans to detect
- Reversibility + verification gate pattern means irreversible-tool support (v2) doesn't require a contract rewrite — just adapter additions
- Conventions (idempotency, timeout context inheritance, redaction) are documented in one place

Negative

- Two shapes to maintain (boundary `AgentResponse` + internal `task_file`); risk of drift
- Verification gate is conceptually defined but functionally inert in v1 (no irreversible tools in fakes); first real test happens in v2
- task_id hierarchy is defined but functionally degenerate in v1 (every id is `<run>.N.0`)

Neutral

- The agent's serialisation layer must explicitly convert `task_file` → `AgentResponse` at the boundary
- Cost tracking lives in a separate `RunCost` record (see classes refactor) referenced from `task_file`, not embedded

---

Open questions / follow-ups

- **Trace format** (ADR 0006) — pending discussion with Ronin (OpenTelemetry vs custom JSON)
- **Multi-phase task modeling** (versioned task_File vs sub-task chain) — deferred to v2 per DEFERRED.md
- **Sensitivity-based verification gate** (verify on reversible-but-sensitive tools) — deferred to v2
- `_contains` **agent-emit support** — deferred to v2 conversation with Ronin

---

References

- Ronin's `AgentResponse` and `AgentToolCall` definitions: `src/baymax/eval/scorer.py` on branch `feat/phase1-scenario-loader`
- Ronin's scenario schema: `src/baymax/eval/schemas/scenario.schema.json`
- Ronin's scoring rules: `docs/SCORING.md` on branch `feat/phase1-scenario-loader`
- Fake adapters surface area: `src/baymax/tools/FAKE_ADAPTERS.md`
- docs/DEFERRED.md for v2/v3 deferrals tied to this contract
- ADR 0009 for reversibility + undo semantics
- docs/WRITEUP_INFO.md for the broader project thesis

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.